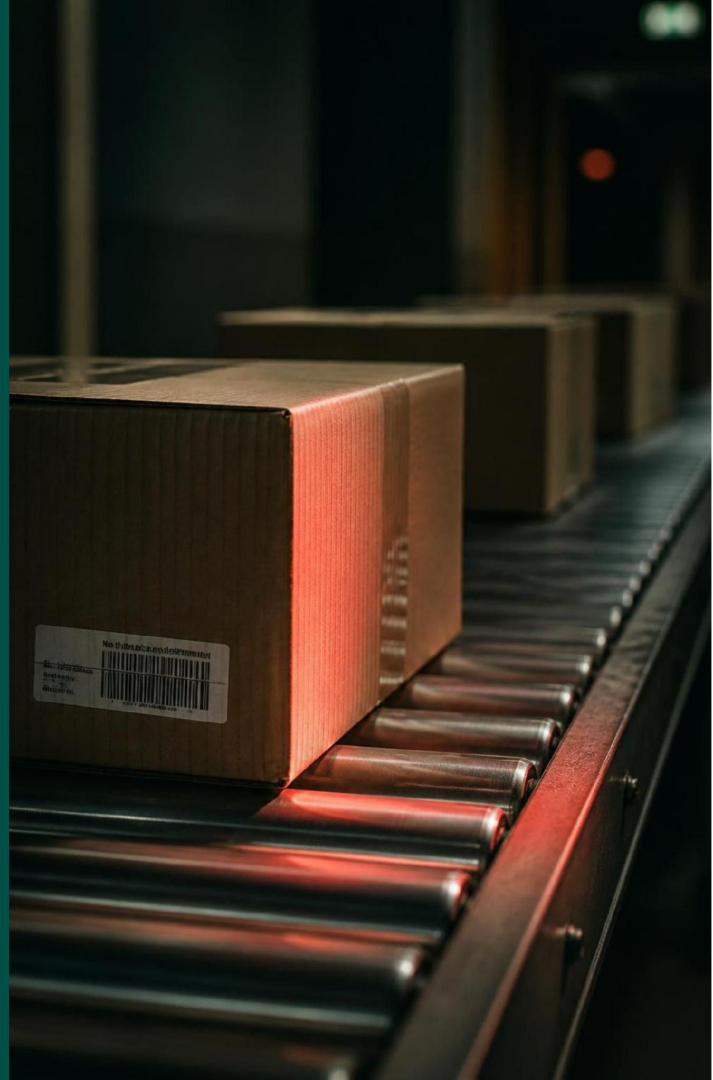
Platform Engineering for Resilient Retail Supply Chains with Event-Driven Design

By: Nidhin Jose



Resilience is a platform capability, not a broker install

- Orders cross storefront, warehouse, store, transport and delivery. The customer sees one promise.
- Event contracts, ownership, replay policies and visibility create resilience.
- Make reliable event-driven development the normal path for product teams.



Each Domain Publishes Only the Facts It Owns

Order Management

Order lifecycle and customer promise.

Inventory

Stock availability, reservations, adjustments.

Logistics

Shipment planning, carrier handoffs, delivery milestones.

Catalog/Pricing

Product information and pricing ownership.

SUCCESS PATH

OrderPlaced → StockReserved → ShipmentDispatched → DeliveryConfirmed

REJECTED BRANCH

OrderPlaced → ReservationRejected

Inventory emits StockReserved or ReservationRejected; each domain owns only the events for its own facts.

Commands can be rejected; events record what happened

Command

Requests an action: ReserveStock asks inventory to reserve stock for an order.

Inventory validates the request and decides the outcome.

Event

Records a completed outcome: StockReserved or ReservationRejected.

Events are facts other services can react to.

- Confusing them creates hidden coupling.
- Domain events express business intent, not table updates.

Kafka Decouples Producers from Consumers

Order Management Service

Publishes OrderPlaced and other order events.

→

Apache Kafka

Topics: orders, inventory, logistics. Partition keys follow each domain's ordering requirements, such as order ID for order events. Retained and replayable.

→

Inventory Service

Consumer group: inventory

Logistics Service

Consumer group: logistics

Order Status Projection

Consumer group: order-status

- Asynchronous: producers do not wait for consumers; each group reads at its own pace.
- Partition keys preserve per-order ordering; global ordering is not assumed.
- Consumers resume from committed offsets after interruptions. Some events may be reprocessed, so consumers must handle duplicates safely.

Kafka is event transport and retention, not a replacement for domain modeling.

The transactional outbox closes the publish gap

Commit then fail to publish, or publish then fail to commit.

Service + DB	Publisher	Kafka
Commit change and outbox event One local transaction updates the database and writes the event to the outbox.	**Publisher reads the outbox** A separate publisher reads the record and delivers the event to Kafka.	**Consumers process independently** Duplicate delivery remains possible; consumers must be idempotent.

The outbox does not eliminate duplicates. Handle retries, stalled records and retention.

Assume at-least-once delivery and classify failures

Retail Example: Duplicate Event Delivery. A StockReserved event may be delivered more than once. Consumers must avoid repeating downstream business actions, such as triggering duplicate fulfillment processing.

The inventory service must also handle repeated ReserveStock commands safely to prevent duplicate inventory reservations.

Stable IDs, idempotent consumers

Repeated deliveries cause no repeated business effect.

Temporary outage

Bounded retry and backoff.

Invalid contract or impossible state

Investigate. Don't retry forever.

Dead-letter queues

Owners, alerts, inspection tooling and safe reprocessing.

Event sourcing: use it where audit and reconstruction pay off

- Ordinary publishing announces changes while state lives in a table. Event sourcing makes the events the record: receipts, reservations, releases and adjustments.

- Invariants must still stop concurrent reservations exceeding stock.

- Costs include immutability, schema evolution, replay, snapshots, retention and corrections.

- Replays must not repeat charges or carrier bookings.

CQRS gives each audience the read model it needs

Command side

- Validates inventory decisions against authoritative state.
- Applies domain rules and publishes business events.

Events →

`StockReserved`, `ShipmentDispatched`

Projections (query side)

- Storefront availability views.
- Fulfillment dashboards.

Projections may lag; final inventory allocation must be validated by the inventory domain. Introduce CQRS selectively: separate models add monitoring and debugging work.

Sagas coordinate fulfillment without a shared transaction

SUCCESS PATH

Stock reservation → Payment authorization → Shipment dispatch

CONDITIONAL COMPENSATION PATHS

Compensation depends on the failure stage; completed business actions may need reversal or recovery.

Stock Reservation		Payment Authorization		Shipment Dispatch
Release reserved inventory.	←	Void the authorization where the payment workflow supports it.	←	Reroute, cancel where possible, or resolve manually.

Compensation is not a database rollback; external actions may need new business operations to reverse.

- Choreography keeps autonomy; orchestration gives visibility. Define states, timeouts, retries, correlation IDs and escalation.

Ordering is scoped, and consistency is a contract

Per partition or aggregate.
Not globally.

- Example: Logistics emits ShipmentPrepared on one partition while Inventory's status update arrives on another, so a projection may apply ShipmentPrepared first.
- Handle with transition rules, version fields, buffering or reconciliation.
- Specify which views may lag and how discrepancies are repaired.

Evolve event contracts without breaking consumers

Backward-compatible changes

Explicit versioning policy.

Contract testing

Consumer migration windows.

Removing a field or changing its meaning is a breaking change: coordinate a versioned, migrated rollout.

Governance should protect interoperability without blocking routine improvements.



Paved paths and visibility make the right design the easy one

- **Self-Service Capabilities**

  Kafka topic provisioning; schema registration; access control and secrets management.

- **Observability and Recovery**

  Consumer lag monitoring; retry and dead-letter queue visibility; event correlation; safe replay.

- **Standardized Development**

  Reusable producer and consumer templates; CI/CD integration; automated contract testing.

- **Governance and Security**

  Least-privilege access; data minimization; appropriate retention; clear service and event ownership.

Platform engineering removes repetitive infrastructure work so product teams build reliable event-driven services consistently.

Adopt incrementally and review failure modes

1

Select a bounded business event

Start with order acceptance or shipment status.

2

Define domain ownership

Producing service, event contract, consumers and business responsibilities.

3

Implement reliable event delivery

Publishing, retries and idempotency mechanisms.

4

Establish observability and recovery

Monitor processing failures, consumer lag and reconciliation needs.

5

Expand gradually

Apply proven patterns to more services while keeping domain boundaries.

Don't use Kafka as a request-response bus: use synchronous APIs when an immediate authoritative decision is required.

Keep business rules in domain services, not platform middleware.

The strongest platform makes correct designs easier to build and operate

- Event-driven architecture supports loosely coupled retail services and asynchronous workflows.
- Reliable publishing, idempotency and appropriate consistency models improve resilience.
- Kafka, CQRS, event sourcing and Sagas solve different problems; choose them on actual requirements.
- Platform engineering provides reusable governance, observability, recovery and service evolution.
- Incremental adoption helps teams manage complexity and validate decisions.

Start with one bounded event, establish reliable platform patterns, and expand through validated architectural decisions.

Thank you

Nidhin Jose